圆形识别-彩色图

圆形识别-彩色图

- 例程效果
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置霍夫圆检测参数
    - 图像处理与霍夫圆检测
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/4.rgb888_find_circles.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的霍夫圆检测



代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 导入摄像头接口 / Camera interface
from media.display import *   # 导入显示接口 / Display interface
from media.media import *     # 导入媒体资源管理器 / Media manager
import _thread
import cv_lite                # 导入 cv_lite 扩展模块 / cv_lite extension
import ulab.numpy as np       # MicroPython 类 NumPy 库
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置 RGB888 像素格式 / Set RGB888 pixel
format
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。

  > 注：摄像头分辨率设为1280*960可以有效抑制显示效果上的变形，但是这样的代码运行时有概率出现紫屏

- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像中的圆检测）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=100)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=100` 设置图像传输质量为较高水平。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置霍夫圆检测参数

```
dp = 1           # 累加器分辨率与图像分辨率的反比 / Inverse ratio of accumulator
resolution
minDist = 30     # 检测到的圆心最小距离 / Minimum distance between detected centers
param1 = 80      # Canny边缘检测高阈值 / Higher threshold for Canny edge detection
param2 = 20      # 霍夫变换圆心检测阈值 / Threshold for center detection in
accumulator
minRadius = 10   # 检测圆最小半径 / Minimum circle radius
maxRadius = 50   # 检测圆最大半径 / Maximum circle radius
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

- `dp`：累加器分辨率比，值为 1 表示与图像分辨率相同，值越大检测速度越快但精度可能降低。
- `minDist`：检测到的圆心之间的最小距离，用于避免重复检测同一圆。
- `param1`：Canny 边缘检测的高阈值，用于提取图像边缘，值越大检测到的边缘越少。
- `param2`：霍夫变换的圆心检测阈值，值越小检测到的圆越多（但可能包含噪声）。
- `minRadius` 和 `maxRadius`：限制检测圆的半径范围，仅检测符合此范围的圆。
- `clock`：用于计算帧率。

图像处理与霍夫圆检测

```
while True:
    clock.tick()

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用

    # 调用 cv_lite 扩展的霍夫圆检测函数，返回圆参数列表 [x, y, r, ...]
    circles = cv_lite.rgb888_find_circles(
        image_shape, img_np, dp, minDist, param1, param2, minRadius, maxRadius
    )

    # 遍历检测到的圆形，绘制圆形框
    for i in range(0, len(circles), 3):
        x = circles[i]
        y = circles[i + 1]
        r = circles[i + 2]
        img.draw_circle(x, y, r, color=(255, 0, 0), thickness=2)  # 红色圆圈

    # 显示带有检测圆的图像 / Display image with circles drawn
    Display.show_image(img)

    # 垃圾回收 / Garbage collect
    gc.collect()

    # 打印帧率 / Print FPS
    print("findcircles:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **霍夫圆检测**：调用 `cv_lite.rgb888_find_circles()` 函数进行圆检测，返回检测到的圆信息列表（每个圆占用 3 个元素：`[x, y, r]`，即圆心坐标和半径）。

- **绘制圆形**：遍历检测到的每个圆，在图像上绘制一个红色圆形轮廓（`color=(255, 0, 0)`）标注目标区域。
- **显示图像**：将处理后的图像显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `dp`：调整累加器分辨率比，值越大检测速度越快，但可能错过一些圆，建议从 1 开始测试。
- `minDist`：根据圆的大小和分布调整，若圆之间距离较近，可减小此值；若检测到重复圆，可增加此值。
- `param1`：Canny 边缘检测高阈值，值越大检测到的边缘越少，若环境噪声多可适当增加此值。
- `param2`：霍夫变换阈值，值越小检测到的圆越多，但可能包含噪声；值越大检测更严格，但可能漏检。
- `minRadius` **和** `maxRadius`：根据目标圆的大小调整范围，确保只检测符合预期的圆。